

Case Study: The Personalized Job-Search Agent

How autonomous AI agents turned 30+ hours of weekly screening work into a 20-minute morning review

The Challenge

An active executive job search generates a relentless stream of work: dozens of postings arrive daily across LinkedIn, Indeed, and company career sites. Each one must be read in full, compared against experience and salary expectations, checked against roles already reviewed, and logged. Done by hand, that is 15–20 minutes per posting — hours of repetitive evaluation every day, prone to fatigue and inconsistency, stealing time from what actually matters: interviews, networking, and preparation.

This is the same problem every business faces with leads, invoices, support tickets, or vendor quotes: a daily flood of documents that someone has to read, judge, and record.

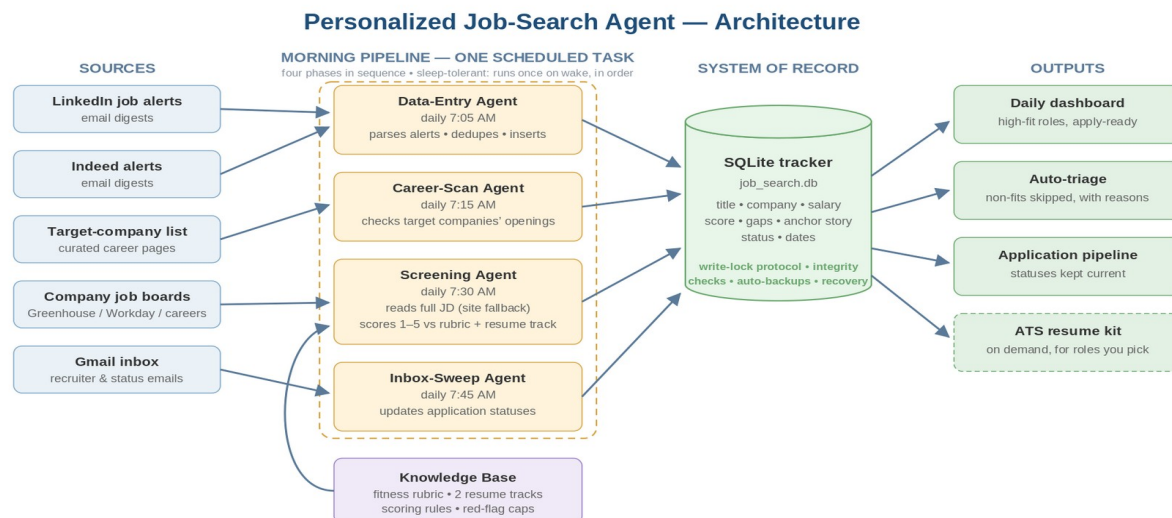
The Solution

A pipeline of scheduled AI agents that runs unattended every morning:

- **Data-Entry Agent (7:05 AM):** parses overnight job-alert emails from LinkedIn and Indeed, de-duplicates against everything already tracked, and inserts new roles into a SQLite database.
- **Career-Scan Agent (7:15 AM):** proactively checks a curated list of target companies' own career pages — catching roles that never hit the job boards — and feeds new matches into the same pipeline.
- **Screening Agent (7:30 AM):** opens each posting, reads the full job description — automatically falling back to the employer's own careers site when LinkedIn withholds content — and scores it 1–5 against a personalized fitness rubric and the correct of two resume versions. It records salary, employment type, key gaps, and the strongest interview “anchor story” for each role.
- **Inbox-Sweep Agent (7:45 AM):** reads recruiter and application emails and keeps every application's status current in the tracker.
- **Resume Tailor (on demand):** turns any chosen high-fit role into an application kit — an ATS-optimized resume variant built from the base resume, honestly mirroring the posting's keywords and leading with the pre-selected anchor story. Roles are picked by a human; nothing is fabricated.

The agents execute as a single sequenced pipeline — if the machine was asleep at 7 AM, the whole sequence runs once on wake, still in order, making schedule collisions impossible by construction.

The human's job shrinks to a morning review of a **personal dashboard** — rebuilt daily from the database — listing high-fit, not-yet-applied roles, each with two actions: generate the ATS resume kit, or jump straight to the application page.



Results — First 7 Days (June 26 – July 2, 2026)

Metric	Result
Roles ingested and fully scored	145 (93 LinkedIn, 44 Indeed, 8 company sites)
Auto-triaged out, with documented reasons	99 roles (68%) — never had to read them
High-fit roles (score 4–5) surfaced	24
Applications submitted and tracked	19, statuses kept current automatically
Manual screening effort replaced	≈35–45 hours (at 15–20 min/posting)
Human time required	≈20 minutes/day reviewing the shortlist
Apply-ready pipeline	Daily dashboard of high-fit open roles with one-click ATS resume kits

Consistency as a feature: every score follows the same written rubric, every skipped role carries a reason, and the system never fabricates data — salary is recorded only when the posting states it, and anything scored from partial information is flagged PROVISIONAL for human review.

What This Pattern Means for a Business

The job-search agent is one instance of a general pattern: **ingest** → **evaluate against your rules** → **record** → **surface what deserves attention**. The same pipeline, re-pointed, becomes:

- Lead scoring and routing — inbound inquiries read, qualified against your ideal-customer profile, and prioritized before the sales call.
- Invoice and PO triage — documents parsed, matched, exceptions flagged.

- Resume screening — applicants scored against the role's actual requirements, consistently.
- Vendor quote comparison — proposals normalized and scored against your criteria.
- Support-ticket triage — issues classified, prioritized, and routed with reasons.

If your team repeatedly reads, judges, and logs anything, this pattern applies.

Technical Appendix

Stack

- Claude agentic AI (Anthropic Agent SDK) with scheduled task execution — four sequenced daily agents (data entry 7:05, career scan 7:15, screening 7:30, inbox sweep 7:45), each guarded by the write-lock protocol.
- SQLite as the system of record; Python for data handling; browser automation for reading postings; Gmail integration for alert parsing and status sweeps.

Design decisions

- **Rubric as single source of truth.** The fitness rubric is a versioned markdown file re-read on every run, so scoring policy changes without touching code. Red-flag rules cap scores only when a disqualifier is a required (not preferred) qualification.
- **Dual resume tracks.** Director-track and TPM-track roles are scored against different resumes; the agent selects the track from the job title.
- **Anti-throttling fallback.** LinkedIn soft-throttles description rendering after a few rapid loads. The agent detects the skeleton state and pivots to the employer's Greenhouse/Workday/careers posting — usually a better source for salary and exact requirements — recording both URLs.
- **Honesty rules.** No fabricated salaries, dates, or contacts; partial-information scores are flagged PROVISIONAL with what would change the score.
- **Idempotent writes.** De-duplication on application URL means re-runs update rather than duplicate.
- **Human-in-the-loop tailoring.** Resume kits are generated on demand for roles the human selects — a deliberate design choice: screening scales to dozens of roles per day, but anything that represents the candidate gets a review step. The daily-rebuilt dashboard bridges the two: fresh data, one-click actions.

Reliability engineering

The database lives on a mounted filesystem where SQLite's native locking does not work, and multiple writers (two scheduled agents, on-demand sweeps, manual edits) share it. An early concurrency collision corrupted the database — and became the system's best feature:

- **Recovery:** a purpose-built salvage tool parses SQLite b-tree pages directly and recovered 124 of 124 rows from the corrupted file.
- **Prevention:** a write-lock protocol with stale-lock detection now serializes all writers; every write goes through a copy-out/verify/copy-back procedure with PRAGMA integrity checks before and after, timestamped backups, and a read-back settle gate that confirms changes actually persisted.
- **Result:** zero data loss since; the protocol is documented so human and AI writers follow identical rules.

This mirrors production operations discipline — locking, integrity gates, backups, and recovery runbooks — applied to a personal automation, and it is the level of care client systems receive.